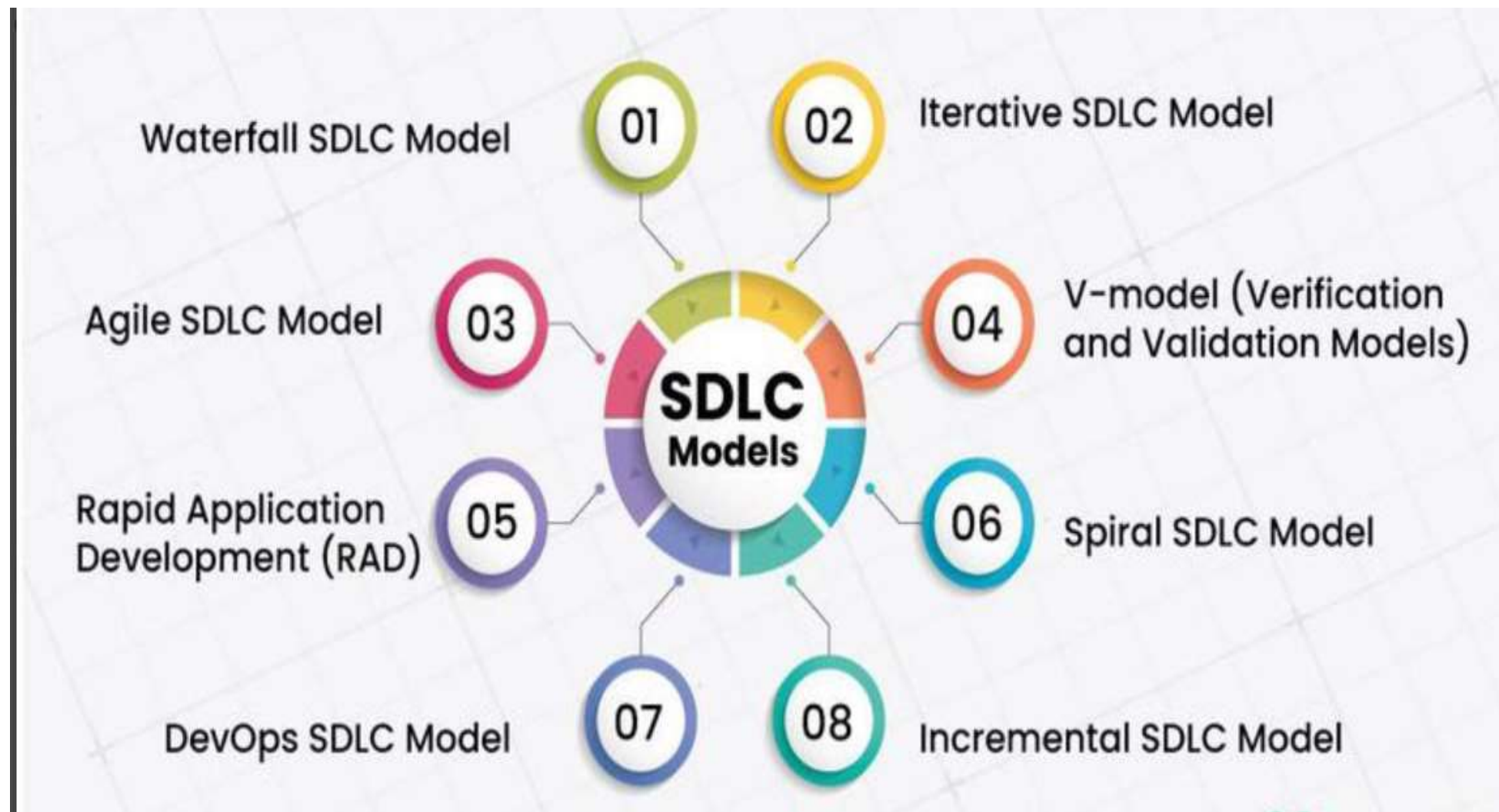


Software Development Models - SDLC Models

- **Definition:**

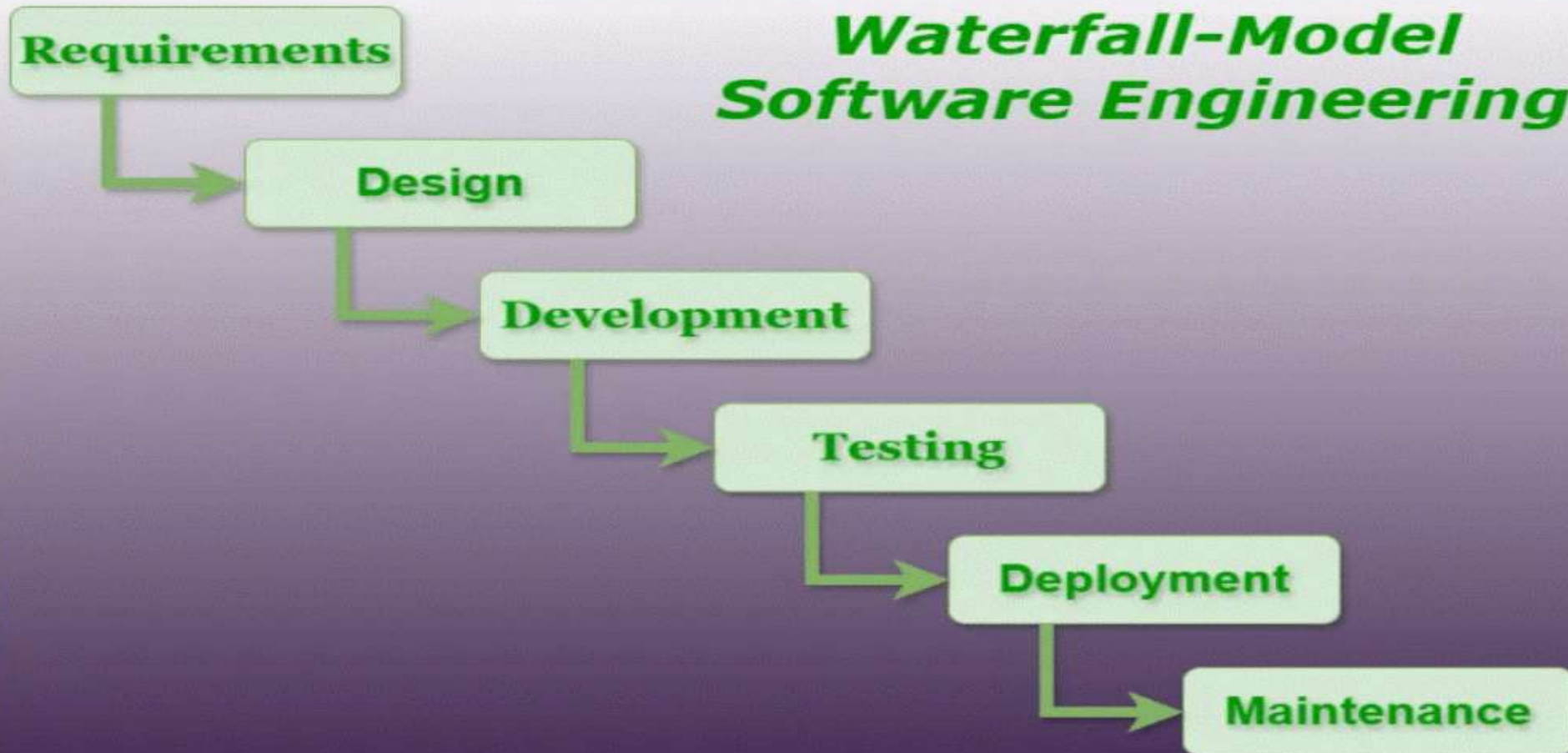
- Software Development Life Cycle (SDLC) models are **frameworks or methodologies** that guide the process of developing software applications from **planning and requirements gathering to deployment and maintenance.**
- Each SDLC model follows a different approach to organizing and managing the phases of software development.



1. Waterfall model

- The Waterfall model is one of the oldest and most straightforward approaches to software development.
- The **Waterfall Model** is a **linear and sequential Software Development Life Cycle (SDLC)** model.
- In this model, each phase of software development is completed before moving to the next phase. The process flows in one direction, like a **waterfall**, making it simple, structured, and easy to understand.

Waterfall-Model Software Engineering



Phases of Waterfall SDLC Models

1. Requirement Analysis

- This is the **first phase** of the Waterfall Model. The development team meets with the client or stakeholders to gather and analyze all the software requirements. The goal is to clearly understand what the customer wants and define the project's scope and objectives.

Activities:

- Gather user requirements.
- Analyze project feasibility.
- Prepare the Software Requirement Specification (SRS) document.

2. System Design

- After the requirements are finalized, the **design phase** begins. In this phase, the development team creates a detailed plan for building the software.

Activities:

- Design the software architecture.
- Design the database and user interface (UI).
- Define system components and modules

3. Development (Implementation)

- In this phase, developers write the source code according to the design specifications. Each module is developed and tested individually to ensure it works correctly.

Activities:

- Write the program code.
- Develop software modules.
- Perform **Unit Testing** on each module.

4. Testing

- After the software is developed, it is tested to ensure that it meets all the specified requirements and is free from defects or errors.

Activities:

- Perform integration testing.
- Conduct system testing.
- Carry out user acceptance testing (UAT) .
- Identify and fix bugs.

5. Deployment

- Once the software has been successfully tested and approved, it is released to the users or installed in the production environment for actual use.

Activities:

- Install the software.
- Configure the production environment.
- Release the application to users.
- Perform final verification (smoke testing).

6. Maintenance

- Maintenance is the final phase of the Waterfall Model. After deployment, the software is monitored and maintained to ensure it continues to work correctly. Any issues are fixed, and updates or enhancements are made as needed.

Activities:

- Fix bugs and errors.
- Improve software performance.
- Update the software based on user feedback.
- Add minor enhancements and security updates

Advantages of the Waterfall Model:

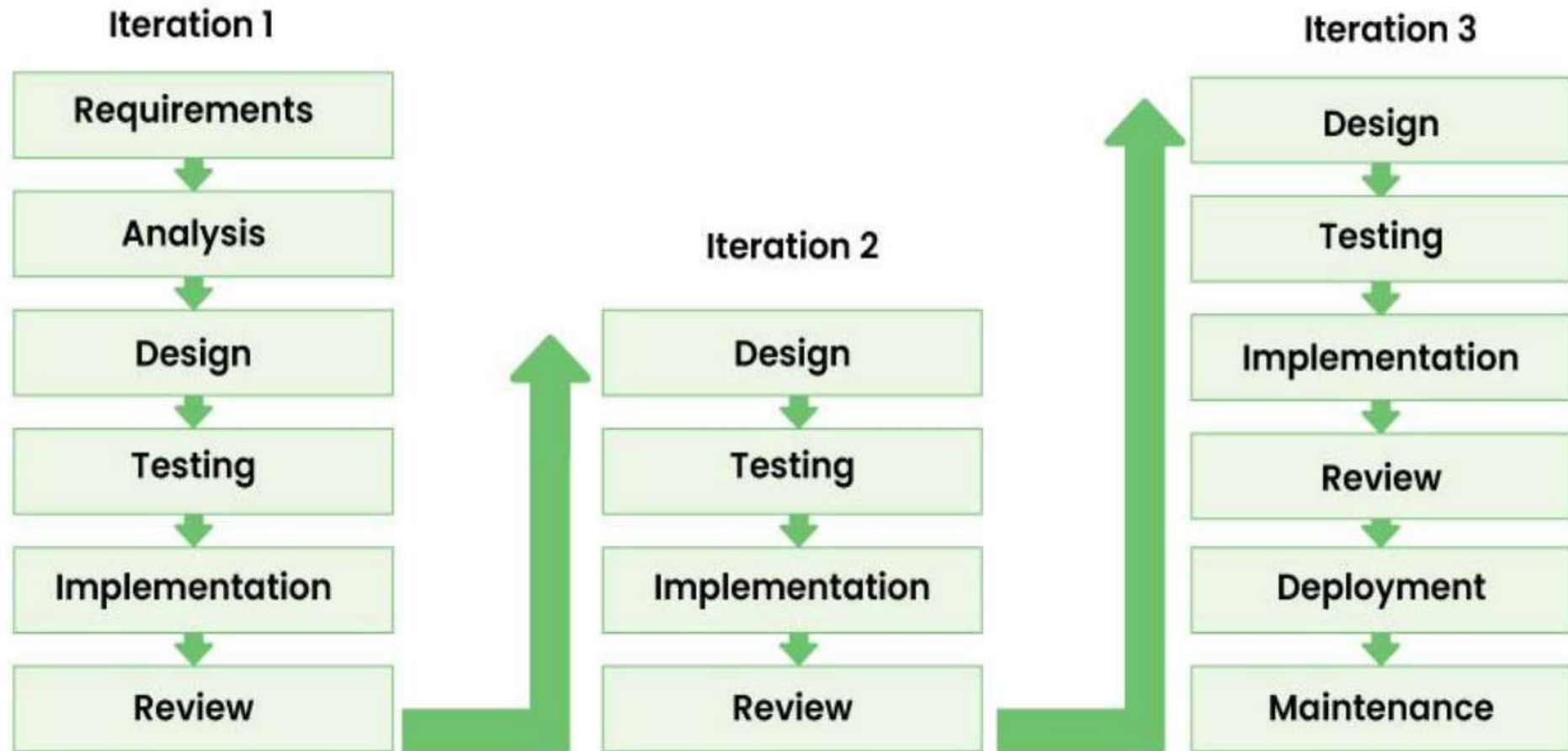
- Simple and easy to understand.
- Provides clear documentation at every phase.
- Suitable for projects with stable and fixed requirements.
- Offers better predictability of project schedule, cost, and deliverables.

Disadvantages of the Waterfall Model:

- Difficult to accommodate changes once development begins.
- Testing is performed only after coding, so defects are identified late.
- Limited customer involvement after the requirements phase.
- Does not support prototyping or early user feedback.

2. Iterative SDLC Models

- The **Iterative SDLC Model** is a software development methodology in which the software is developed **through repeated cycles (iterations)**. Instead of building the entire system at once, the project is developed in **small parts**, and each iteration improves the software based on testing and user feedback.
- The Iterative Model is **flexible and efficient** because it allows continuous improvement, early testing, and easy adaptation to changing requirements.



Iteration 1 – Initial Version

- This is the first development cycle where the basic version of the software is created.

Steps:

- **Requirements** – Gather the initial requirements from the client.
- **Analysis** – Analyze the requirements and determine feasibility.
- **Design** – Create the system design and architecture.
- **Testing** – Test the developed features for defects.
- **Implementation** – Develop (code) the first version of the software.

Iteration 2 – Improved Version

- Based on the feedback from Iteration 1, improvements and additional features are added.

Steps:

- Design
- Testing
- Implementation
- Review

Iteration 3 – Final Version

- The software is further refined and prepared for release.

Steps:

- **Design** – Design the remaining features.
- **Testing** – Test the new features and overall system.
- **Implementation** – Complete the remaining development.
- **Review** – Final evaluation and customer approval.
- **Deployment** – Release the software to users.

Advantages of the Iterative SDLC Model

- Faster delivery of working software through incremental development.
- Flexible enough to accommodate changing requirements.
- Better software quality due to continuous testing and improvements.
- Regular stakeholder feedback ensures the final product meets user and business requirements.

Disadvantage

Higher Cost

Repeated development and testing increase project cost.

Time-Consuming

Multiple iterations may take longer to complete.

Requires Skilled Team

Needs experienced developers and managers.

Scope Creep

Frequent changes may expand the project scope.

Complex Management

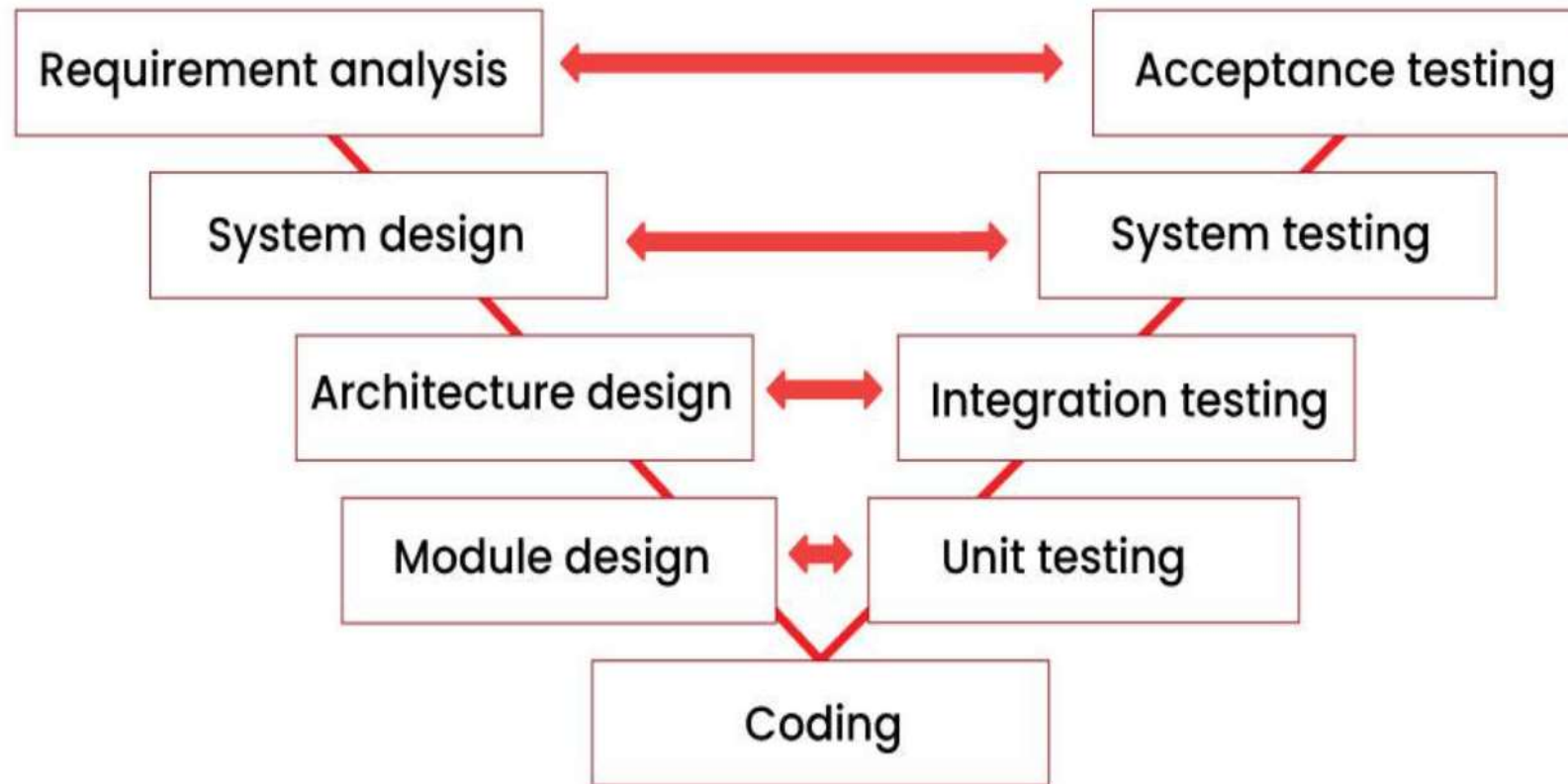
Managing multiple iterations is more difficult.

More Documentation

Documentation is needed after each iteration.

V-Model (Verification and Validation Model) in SDLC

- The **V-Model (Verification and Validation Model)** is an **extension of the Waterfall Model**. It follows a **sequential development process**, but unlike the Waterfall Model, **each development phase has a corresponding testing phase**.
- The development activities are on the **left side** of the V, and the testing activities are on the **right side**, forming a **V-shaped structure**.



Key Principles of the V-Model

1. Parallel Development and Testing

- Unlike the Waterfall Model, the **V-Model performs development and testing in parallel.** Every development phase has a corresponding testing phase, which helps detect defects early and improves software quality.

Example:

- Requirement Analysis ↔ Acceptance Testing
- System Design ↔ System Testing
- High-Level Design ↔ Integration Testing
- Low-Level Design ↔ Unit Testing

2. Verification and Validation

- The V-Model gives equal importance to **Verification** and **Validation**.
- **Verification** checks whether the software is being developed **correctly** according to the requirements and design.
 - **Activities:** Requirement review, design review, code review.
- **Validation** checks whether the final software **meets the customer's needs**.
 - **Activities:** Unit Testing, Integration Testing, System Testing, Acceptance Testing.

3. Traceability

- The V-Model provides **traceability** by creating a direct link between every development phase and its corresponding testing phase. This ensures that all requirements are properly designed, implemented, and tested.

Benefits:

- Better documentation.
- Easy tracking of requirements.
- Ensures complete test coverage.
- Helps identify missing or untested requirements.

4. Early Defect Detection

- Since testing activities are planned from the beginning and performed alongside development, defects are identified and corrected **early**. This reduces the cost and effort required to fix errors later.

Benefits:

- Improves software quality.
- Reduces development cost.
- Saves time.
- Produces a more reliable software product.

Advantages of the V-Model

1. Clear Design and Planning

- The V-Model follows a **well-structured framework** with clearly defined phases. Each stage has specific tasks and deliverables, making project planning and management easier.

2. Early Issue Identification

- Testing activities begin **from the early stages of development**. This helps identify and fix defects before they become major problems, reducing development time and cost.

3. Traceability and Documentation

- The V-Model maintains a **direct relationship between development phases and testing phases**. This ensures complete documentation and makes it easier to track project progress and verify that all requirements are tested.

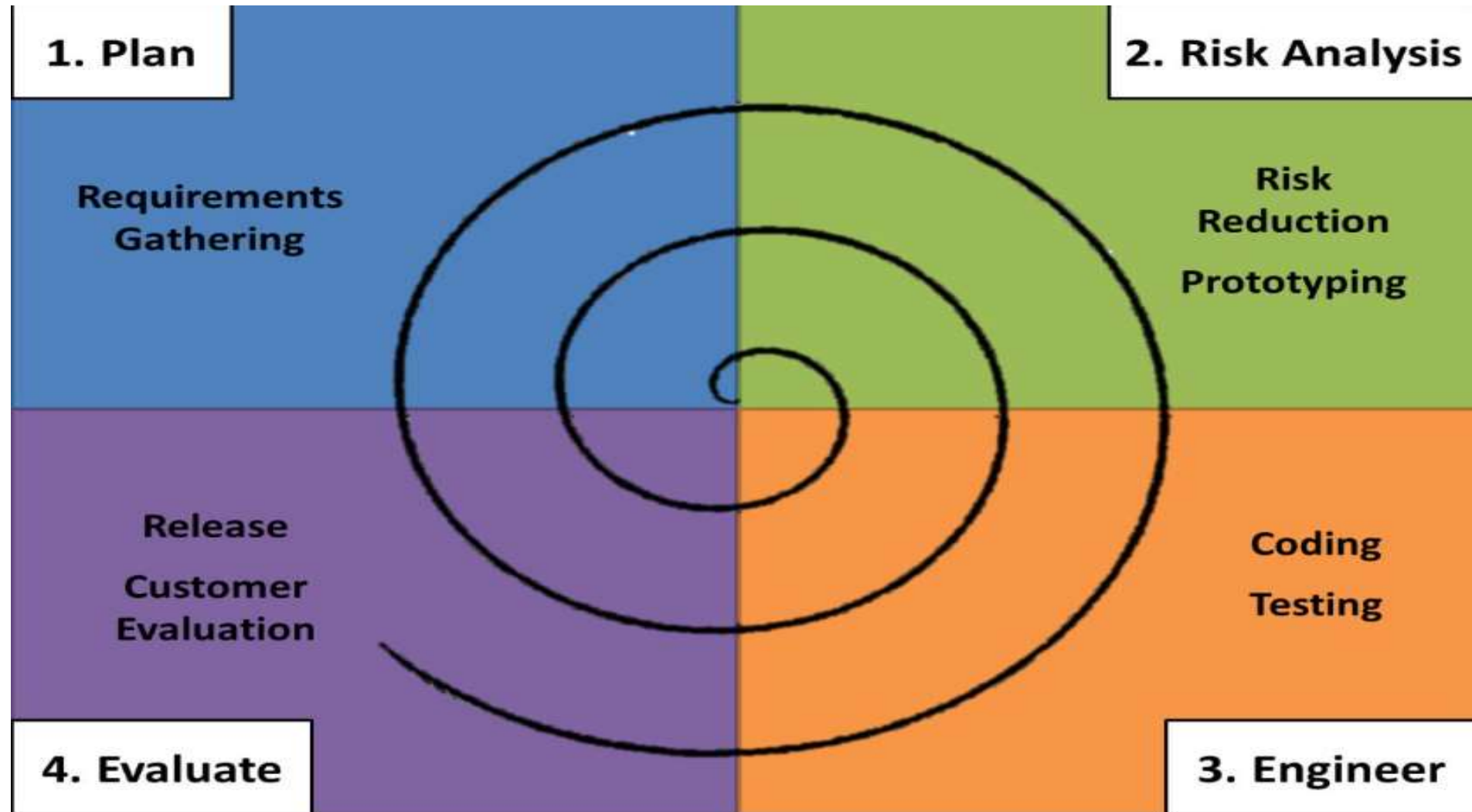
4. Predictability and Control

- The systematic and sequential nature of the V-Model makes the project **more predictable and easier to control**. Stakeholders have a clear understanding of project milestones, timelines, and deliverables.

4. Spiral SDLC Model

- The **Spiral SDLC Model** is a software development model that **combines the iterative approach of the Iterative Model with the systematic phases of the Waterfall Model.**
- Development progresses through a series of **spiral loops**, where each loop represents one phase or iteration of the project.

- The Spiral Model is **risk-driven**, meaning that **risks are identified, analyzed, and addressed during every iteration**. This makes it highly suitable for **large, complex, and high-risk projects**.



Phases of the Spiral Model

- Each loop of the spiral consists of **four phases**:

1. Planning

- Gather customer requirements.
- Define project objectives and constraints.
- Plan the activities for the current iteration.

2. Risk Analysis

- Identify possible risks.
- Analyze technical, financial, and schedule risks.
- Develop strategies to reduce or eliminate

3. Engineering (Development)

- Design, code, and test the software.
- Develop a working version of the product.

4. Evaluation

- the developed software to the customer.
- Collect feedback.
- Decide whether another iteration is needed.
- The cycle continues until the complete software is developed.

Advantages

- Excellent risk management.
- Flexible to changing requirements.
- Continuous customer feedback.
- Suitable for large and complex projects.
- Early detection and resolution of risks.
- Delivers improved software after every iteration.

Disadvantages

- Expensive due to repeated risk analysis.
- Requires experienced project managers.
- Complex to implement and manage.
- Not suitable for small or low-budget projects.
- Development time may increase because of multiple iterations.

5. Incremental Process Model

- The **Incremental Process Model** is a software development approach in which the software is developed and delivered in **small parts called increments**.
- Each increment adds new functionality to the previous version until the complete system is finished.

Phases of Incremental model

01

Requirement Analysis

02

Design & Development

03

Testing

04

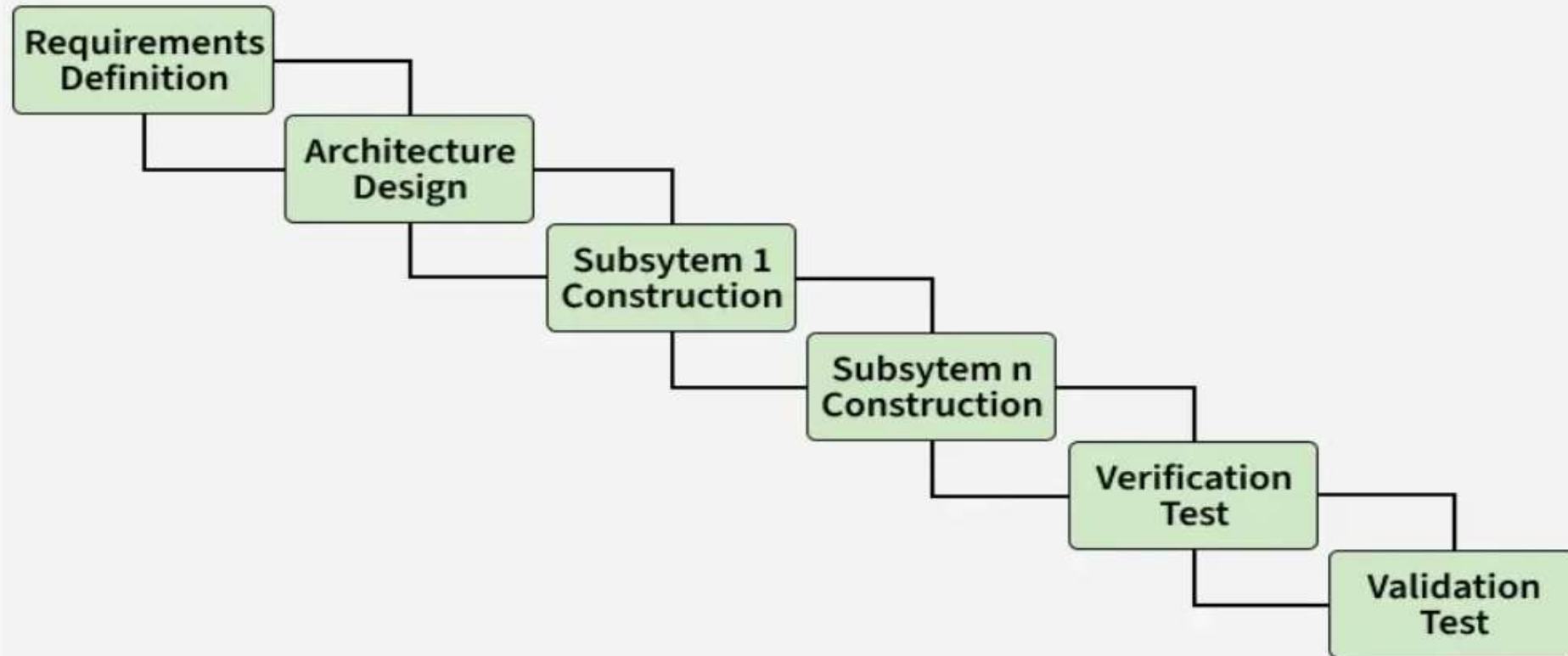
Implementation

- **Requirement Analysis:** Requirements for the current increment are gathered and clearly defined.
- **Design & Development:** The system design is prepared, and new features are developed.
- **Testing:** Each increment is tested to ensure new and existing features work correctly.
- **Implementation:** The tested increment is delivered and integrated with previous versions.

- **Types of Incremental Model**
- The **Incremental Model** has **two main types**, each offering a different approach to developing software in small parts (increments).

1. Staged Delivery Model

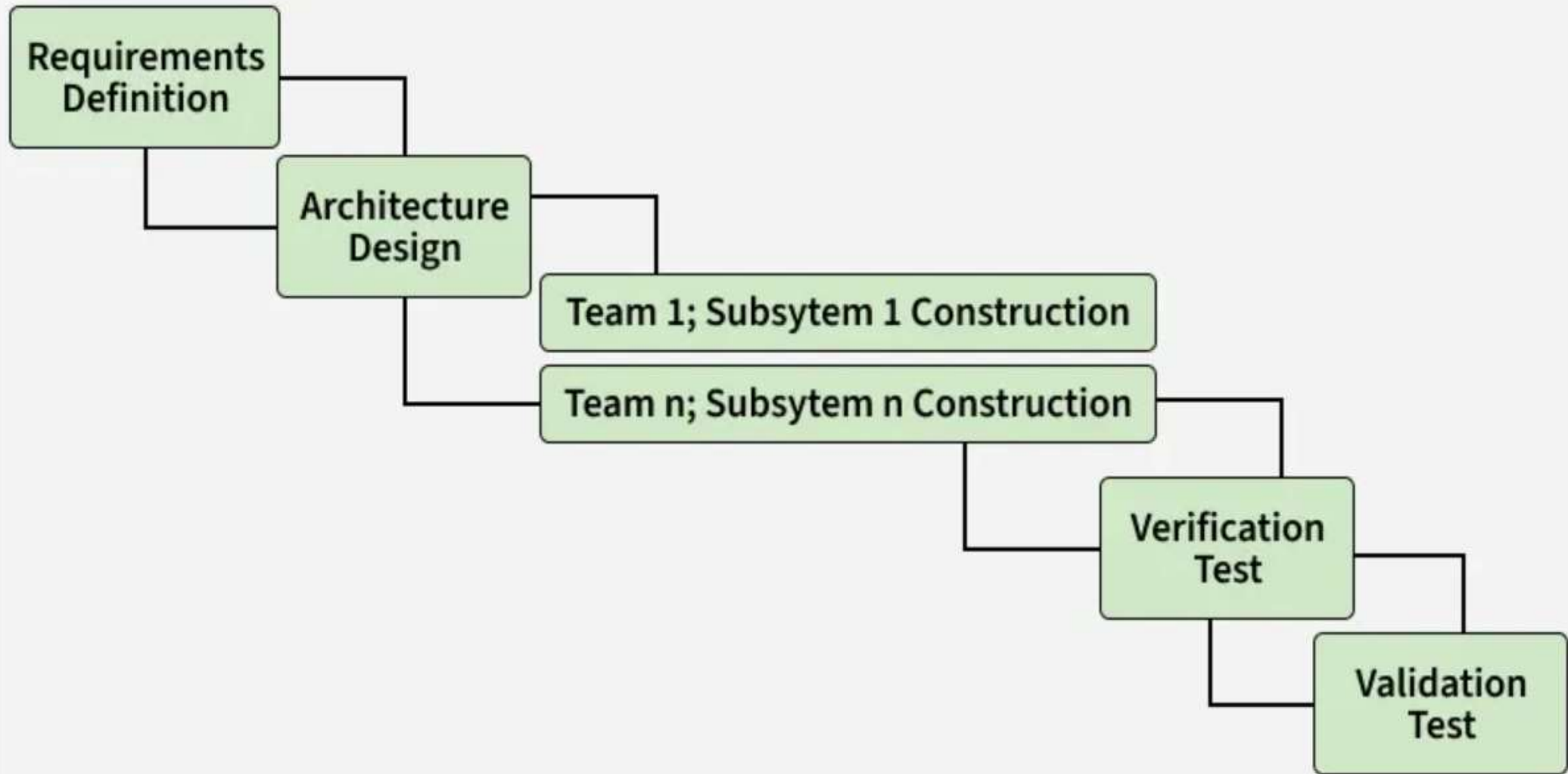
- The **Staged Delivery Model** is an incremental software development approach in which the software is built and delivered in **planned stages**.
- Each stage provides a **working and functional part** of the system. With every new release, more features are added until the complete software is finished.
- This approach allows users to see progress regularly, provide feedback, and identify issues



Staged Delivery Model

- **Parallel Development Model**

- The **Parallel Development Model** is an incremental software development approach in which the system is **divided into multiple modules**.
- After the overall system design is completed, different modules are **developed simultaneously (in parallel)** by different development teams.
- Since multiple teams work at the same time, the overall development time is reduced, making the process faster and more efficient. Once all modules are completed, they are integrated to form the final software system



- **Advantages**

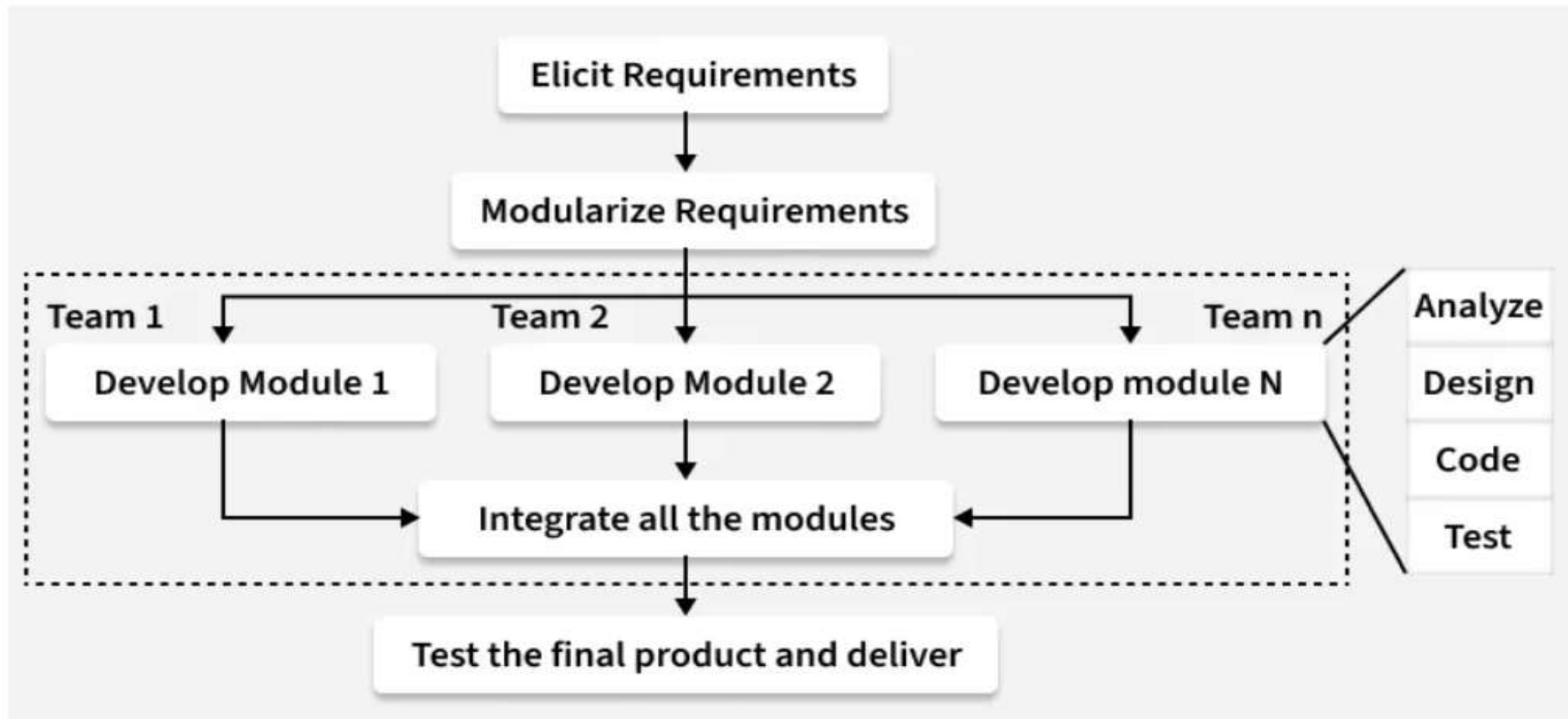
- Early Software Delivery, improves customer satisfaction
- Easy to Manage Changes between increments
- Better Risk Management through early testing
- Cost Control by spreading budget over stages
- Simpler Debugging due to modular development
- High Software Quality through continuous testing

Disadvantages

- Requires Skilled Team and proper planning
- Cost May Increase due to repeated testing and integration
- Design Issues if future requirements are unclear
- Integration Challenges between increments
- Repeated Bug Fixing across multiple increments

6. Rapid Application Development (RAD) model

- The **Rapid Application Development (RAD) Model** is an **iterative and incremental software development model** that focuses on **rapid prototyping, quick development, and continuous user feedback** instead of extensive planning.
- It divides the project into small modules that are developed in parallel within **short development cycles (60–90 days)**.
- Users review each prototype and provide feedback, allowing improvements throughout the development process.



RAD Model

Advantages

- **Fast development:** Reusable components reduce development time.
- **Early customer feedback:** Users see prototypes early and give feedback.
- **Lower development effort:** Fewer developers are needed due to automation.
- **Better quality:** Powerful tools help build quality software quickly.
- **Easy progress tracking:** Development progress is visible in each phase.
- **Flexible to changes:** Changing requirements are easy to handle due to short cycles.
- **High productivity:** Small teams can deliver results faster.

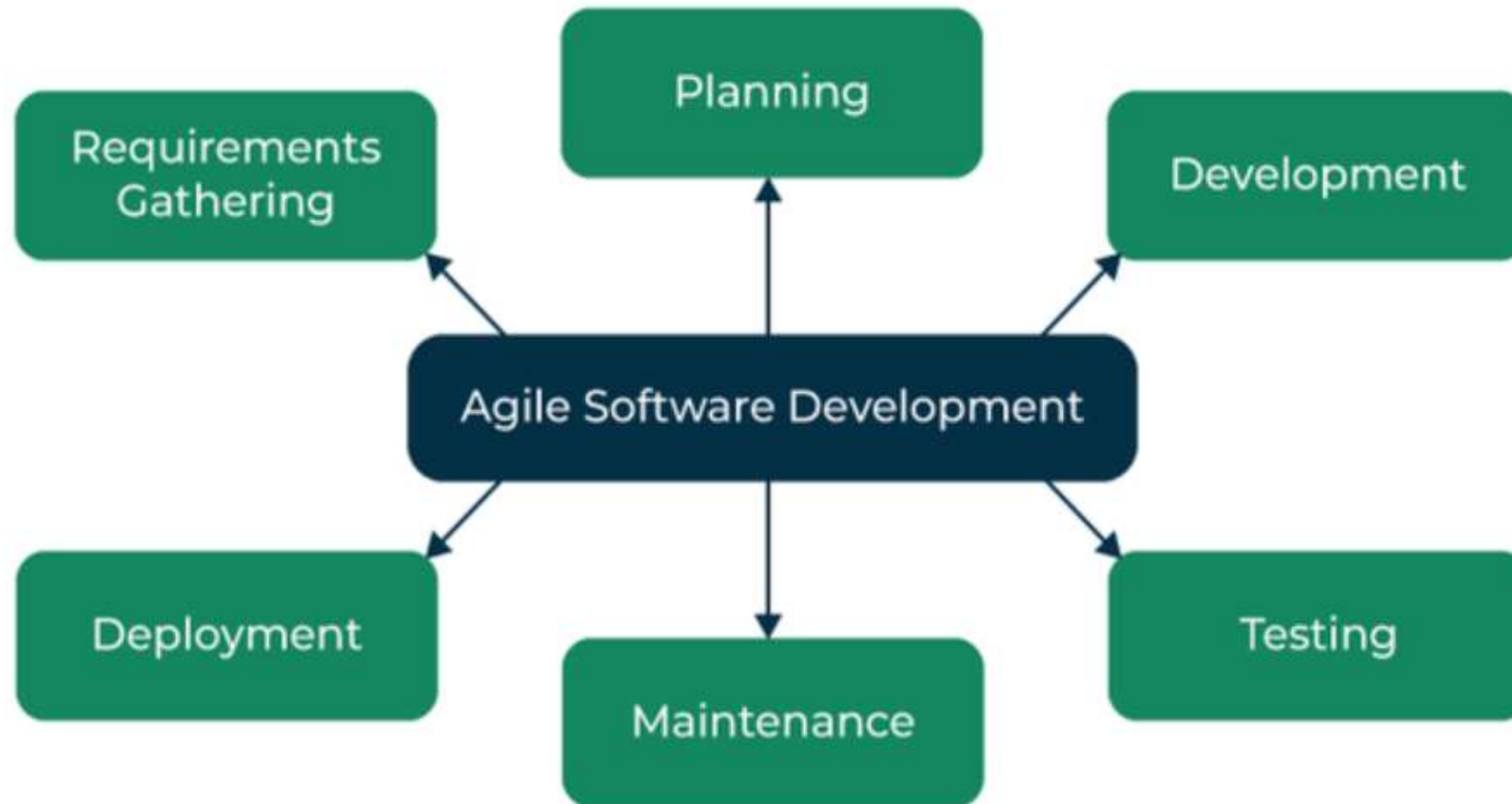
Disadvantages

- **Needs skilled professionals:** Developers must be experienced with advanced tools.
- **Depends on reusable components:** Lack of reusable parts can cause failure.
- **High coordination needed:** Team leader must closely manage developers and customers.
- **Not suitable for all systems:** Systems that cannot be modularized cannot use RAD.

- **Continuous customer involvement required:** Lack of user commitment can fail the project.
- **Costly for small projects:** Tool and automation costs may exceed the budget.
- **Not suitable for every application:** Some projects do not fit RAD structure

7. Agile Software Development

- The **Agile Software Development Model** is a software development methodology that emphasizes **flexibility, collaboration, and customer satisfaction**.
- It follows an **iterative and incremental approach**, where software is developed in small parts called **iterations (or sprints)**.
- Each iteration delivers a **working product quickly**, allowing customers to provide continuous feedback.
- The development team and customer work closely together to ensure the software meets changing requirements and user expectations.



Agile Software Development

Characteristics of Agile Software Development

1. Agile as a Mindset

- Agile is not just a development method; it is a way of thinking and working.
- It focuses on adaptability, teamwork, continuous improvement, and customer satisfaction.
- **Example:** If a customer requests a new feature, the team is ready to adapt instead of following a fixed plan.

2. Quick Response to Change

- Agile welcomes changing requirements, even during development.
- Teams can quickly modify the software based on customer feedback or new business needs.
- This makes Agile suitable for projects where requirements change frequently.
- **Example:** An online shopping app adds a new payment method after customer feedback without waiting for the entire project to finish.

3. Regular Demonstrations

- At the end of every **Sprint (2–4 weeks)**, the team demonstrates the working software to the customer.
- Customers review the progress and provide feedback.
- This ensures that the project is moving in the right direction and meets customer expectations.
- **Benefit:** Problems are identified early, reducing the cost of fixing them later.

4. Cross-Functional Teams

- Agile teams consist of members with different skills, such as **developers, testers, designers, business analysts, and product owners**.
- Team members work together, share knowledge, and communicate regularly.
- The team is **self-organizing**, meaning members decide how to complete the work without constant supervision.
- **Example:** While developers write code, testers test new features, and designers improve the user interface—all working together during the same sprint.

12 Principles of Agile Methodology

01

Customer Satisfaction

02

Changing Requirement

03

Frequent Delivery

04

Promoting Collaboration

05

Motivated Individuals

06

Face to Face Communication

07

Maintain a Constant pace

08

Measure Progress

09

Technical Excellence

10

Simplicity

11

Self organized Teams

12

Continues Improvements

Advantages :

1. Improved Collaboration and Communication
2. Flexibility and Adaptability
3. Better Quality and Reliability
4. Higher Customer Satisfaction

Disadvantages

1. Lack of Predictability
2. Limited Scope Control (Scope Creep)
3. Less Focus on Testing
4. Risk of Team Burnout
5. Lack of Structure